



# Chapter 7 软件实施与测试方法

- 7.1 程序设计语言
- 7.2 编码风格
- 7.3 软件测试原则
- 7.4 白盒测试
- 7.5 黑盒测试
- 7.6 单元测试
- 7.7 集成测试
- 7.8 相关文档规范





# 7.1 程序设计语言

- 广泛应用的程序设计语言

- 汇编语言

- 面向机器的底层语言，直接对计算机硬件操作
    - 开发操作系统、设备驱动程序等

- **BASIC**初学者通用符号指令代码

(Beginner's All - purpose Symbolic Instruction Code)

- 面向过程，早期用于个人计算机编程教学

- **Fortran**(Formula Translator)

- 主要用于科学计算、工程计算等领域



# 7.1 程序设计语言

## • 广泛应用的程序设计语言

### – C / C++ / Objective-C/Swift

- **C:** 广泛应用的编程语言，兼具高级语言的便利性和低级语言对硬件的操作能力，系统软件、嵌入式系统、工具软件等开发
- **C++:** 在 C 语言基础上发展而来的面向对象编程语言，增强了代码的可维护性和可扩展性，用于大型软件系统、游戏开发等
- **Objective - C:** 主要用于苹果操作系统（macOS、iOS 等）的应用开发，是面向对象的编程语言
- **Swift:** 苹果开发用于 iOS、macOS 等，逐渐取代 Objective - C 成为苹果平台开发的主流语言



# 7.1 程序设计语言

- 广泛应用的程序设计语言

- Java

- 跨平台，具有自动内存管理（垃圾回收）等特性，广泛用于企业级应用开发、安卓应用开发等

- .NET系列

- 微软的一套开发框架，支持多种编程语言(如 C# 等)，用于Windows 平台应用开发、Web 应用开发等

- Python

- 数据科学、人工智能、网络爬虫、自动化脚本等众多领域广泛应用

- JSP/ASP/PHP/Perl/HTML.....



# 7.1 程序设计语言

- 程序设计语言的选择
  - 生产率因素
  - 软件应用领域
  - 程序员的知识
  - 用户的要求
  - **CASE**工具支持



## 7.2 编码风格

- 程序内部文档
  - 注释
- 程序标识符
  - 文件标识符、数据标识符、变量标识符、常数标识符
- 程序清单
  - 语句、循环、条件
- 程序语句与编码
  - 表达式、IF、输入、输出



## 7.2 编码风格

- 程序的内部文档
  - 用途：
    - 正确的注释能够帮助读者理解程序
    - 为测试和维护阶段提供明确的指导
  - 注释行的数量：
    - 计入工作量：充数
    - 不计入工作量：不愿写
    - 占整个源程序的1/3到1/2
  - 注释分为：
    - （头部）序言性注释
    - （内部）功能性注释





## 7.2 编码风格

- 序言性注释：置于分，给出程序的整
- 规范
  - (1) 程序标题
  - (2) 有关本模块功能和目的
  - (3) 算法说明
  - (4) 适用说明：
    - ✓ 接口：包括调用形
    - ✓ 有关数据描述：重条件等
  - (5) 模块位置：在哪一个沙
  - (6) 开发简历：模块设计期、有关说明等

```
// Predicts class scores using a trained ELM model.
//
// scores = mexElmPredict( inW, bias, outW, X );
//
// INPUT :
// inW      - input weights matrix (trained model)
// bias     - bias vector (trained model)
// outW     - output weights matrix (trained model)
// X        - samples matrix (vectors in columns)
//
// OUTPUT :
// scores   - prediction scores for samples in matrix X
//
#include "elm1.hpp"

#include "mex.h"
#include "matrix.h"

// input variable order numbers
#define INW 0
#define BIAS 1
#define OUTW 2
#define XDATA 3

// output variable order numbers
#define SCRS 0
```





## 7.2 编码风格

- 功能性注释

- 嵌在源程序体中
- 用以描述一段语句或程序段，解释要“做什么”或是执行了下面的语句会怎么样
- 注意以下几点：
  - ✓ 用于描述一段程序，而不是每一个语句
  - ✓ 用缩进和空行，使程序与注释容易区别
  - ✓ 注释要正确

```
total = amount + total;  
// Add amount to total  
// Add monthly-sales to annual-total  
月销售额计入年度总额
```



## 7.2 编码风格

- 程序的标识符：望文生义

- 程序文件标识符

- 与设计文档一致

- 数据文件标识符

- 变量标识符

- $S=X*Y*Z$ ? 什么意思?
    - $\text{Volume}=\text{length}*\text{width}*\text{height}$
    - Times, total, average, sum
    - 公有变量、私有变量的命名

- 常数标识符

- $\text{Netpay}=\text{grosspay}*(1-0.04-0.0175)$
    - $\text{taxrate}=0.04$
    - $\text{Sectaxrate}=0.0175$
    - $\text{Netpay}=\text{grosspay}*(1-\text{taxrate}-\text{sectaxrate})$



## 7.2 编码风格

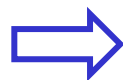
- 程序清单的安排

- 语句序列

- 一行内只写一条语句
    - 同一层次的语句序列写在相同的列上，全部语句的第一个字母要对齐

- 清晰

```
a[i] = a[i] + a[t];  
a[t] = a[i] - a[t];  
a[i] = a[i] - a[t];
```



```
WORK = a[t];  
a[t] = a[i];  
a[i] = WORK;
```

- 循环语句

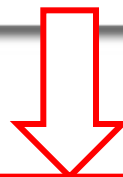
- 循环语句的语句体部分要适当的缩进

- 条件选择语句

- 条件选择语句中的**then**部分和**else**部分，应该写在不同的行上，并且应有适当的缩进

```
a=1; b=2;
```

```
int a, b;
```



```
a=1;  
b=2;
```

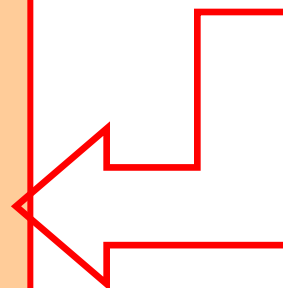
```
int a;  
int b;
```



## 7.2 编码

```
FOR I:=1 TO N - 1 DO BEGIN T:=I;  
FOR J:=I + 1 TO N DO IF A[J] < A[T]  
THEN T:=J; IF T≠I THEN BEGIN  
WORK:=A[T]; A[T]:=A[I];  
A[I]:=WORK; END END;
```

```
FOR I:=1 TO N-1 DO  
BEGIN  
    T:=I;  
    FOR J:=I + 1 TO N DO  
        IF A[J] < A[T] THEN T:=J;  
    IF T≠I THEN  
        BEGIN  
            WORK:=A[T];  
            A[T]:=A[I];  
            A[I]:=WORK;  
        END  
END;  
END;
```





## 7.2 编码风格

- 程序中的语句

- 由表达式构成的语句

- 适当使用括号、空格

- If  $a+b \geq c-d$  then...

- If  $(a+b) \geq (c-d)$  then...

- $A=B*C+3**D/-3*X+Y*4$

- $A=(B*C)+3**D/((-3)*X)+(Y*4)$

- 分解成更小的表达式

- $Part1=B*C$

- $Part2=3**D/((-3)*X)$

- $Part3=(Y*4)$

- $A=Part1+Part2+Part3$



## 7.2 编码风格

```
NORTH=CAT+DOG+ALLIGA  
TOR+HIPPOPOTAMUS+3.14  
159-2.79128
```

- 换行：不能在变量名或常数中间换行



```
NORTH=CAT+DOG+ALLIGATOR+HIPPOPOTAMUS+3.14159-2.79128
```

### —IF语句

- 尽量避免否定的布尔条件

x If **not** A then B else C

✓ If A then C else B

- 尽量避免if ... then if 形式

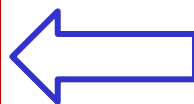
- 避免else return的形式

```
IF ( CHAR >= 'A' ) THEN  
IF ( CHAR <= 'Z' ) THEN  
PRINT "This is a letter."  
ELSE  
PRINT "This is not a letter."
```



```
IF ( CHAR >= 'A' ) and ( CHAR <= 'Z' )
```

```
Int exam (int x)  
{  
    int y=0;  
    if x>0 then y=100*x  
    return y  
}
```



```
Int exam (int x)  
{  
    if x>0 then  
        return 100*x;  
    else  
        return 0  
}
```



## 7.2 编码风格

- 程序中的语句

- 输入语句

- 指明输入种类和取值范围
    - 有明确的提示，识别错误输入
    - 如有必要，进一步确认
    - 合法性检查
    - 批量输入时，使用输入结束标志

- 输出语句

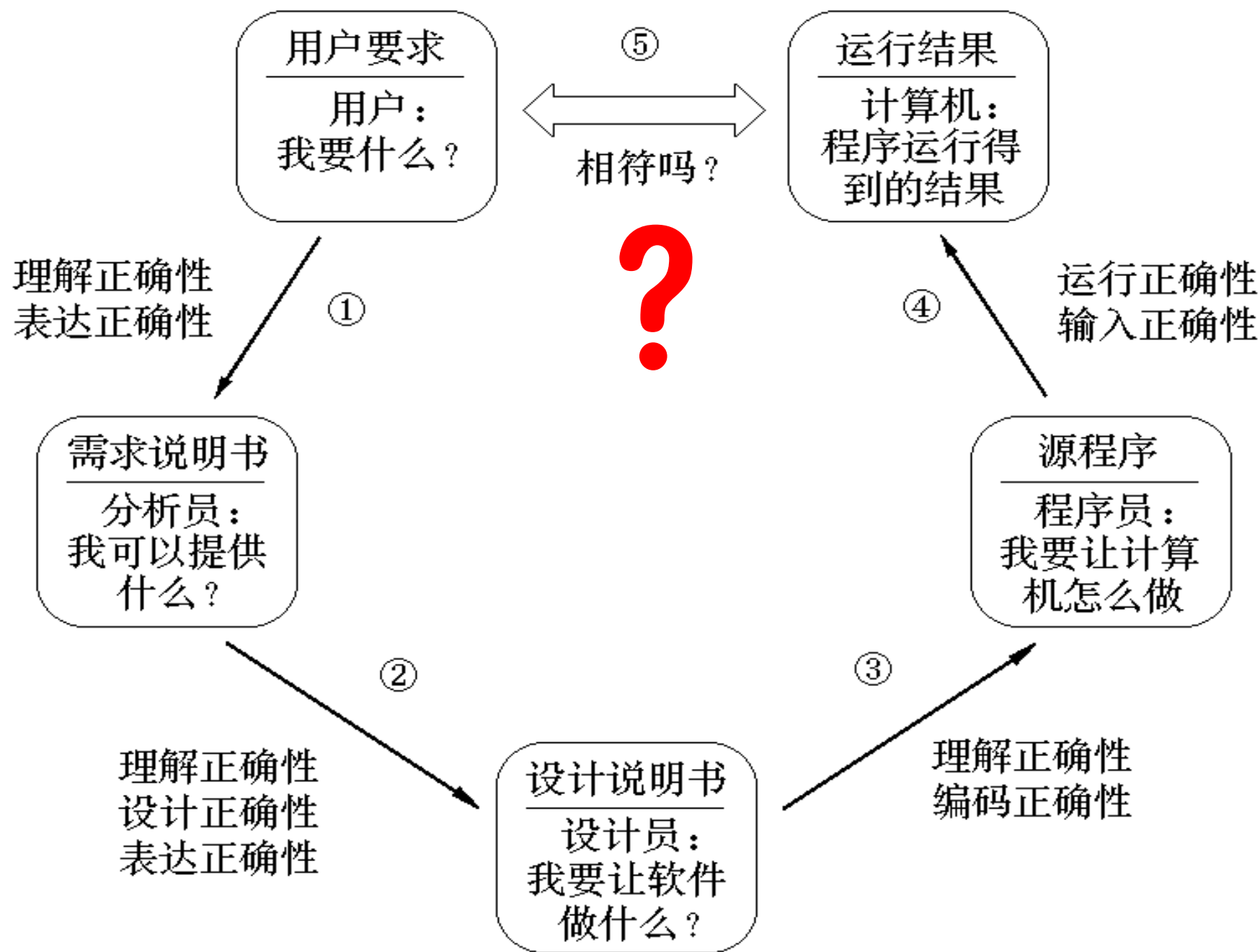
- 加说明信息





## 7.2 软件实施总结

- 编码规范
  - 文件结构、排版、注释、表达式和语句、标识符命名规则、函数的设计、类的设计、内存管理、错误处理...
- 尽可能采用成熟可靠的技术
- 跟踪调试
- 写总结
- 源代码的版本管理





## 7.3 软件测试目的和原则

- 软件测试

- 精心设计一批**测试用例**（即输入数据及其预期的输出结果），并利用这些测试用例去运行程序，以发现程序错误的过程

- 为什么需要测试？

- 1963年，美国飞往火星的火箭爆炸，损失\$10 million。原因：**FORTRAN**循环

- **DO 5 I = 1, 3** 误写为 **DO 5 I = 1.3**

- 软件测试目的：发现程序错误

- **好的测试用例**在于能发现至今未发现的错误
  - **成功的测试**是发现了至今为止尚未发现的错误的测试——Grenford J. Myers 《软件测试的艺术》(The Art of Software Testing)



## 7.3 软件测试目的和原则

- 测试工作量和测试人员
  - 整个软件开发中，测试工作量一般占30%~40%，甚至 $\geq 50\%$
  - 在人命关天的软件（如飞机控制、核反应堆等）中，测试所花费的时间往往是其它软件工程活动时间之和的3~5倍
  - Windows2000开发人员中
    - 项目经理约250人，开发人员约1700人
    - 测试人员约3200人



## 7.3 软件测试目的和原则

- 遵循的原则
  - 所有测试的标准都建立在用户需求之上
  - 所有的需求都是可验证的
  - 测试活动可提前展开
  - 增量测试
  - 穷举所有的测试是不现实的
  - 不要忽略非正常的输入数据
  - 不能忽略回归测试
  - 对问题较多的代码单元，要进行更细致的测试
  - 专业人员测试或委托第三方测试

### Pareto原则

测试所发现错误中的**80%**可能源于程序的**20%**



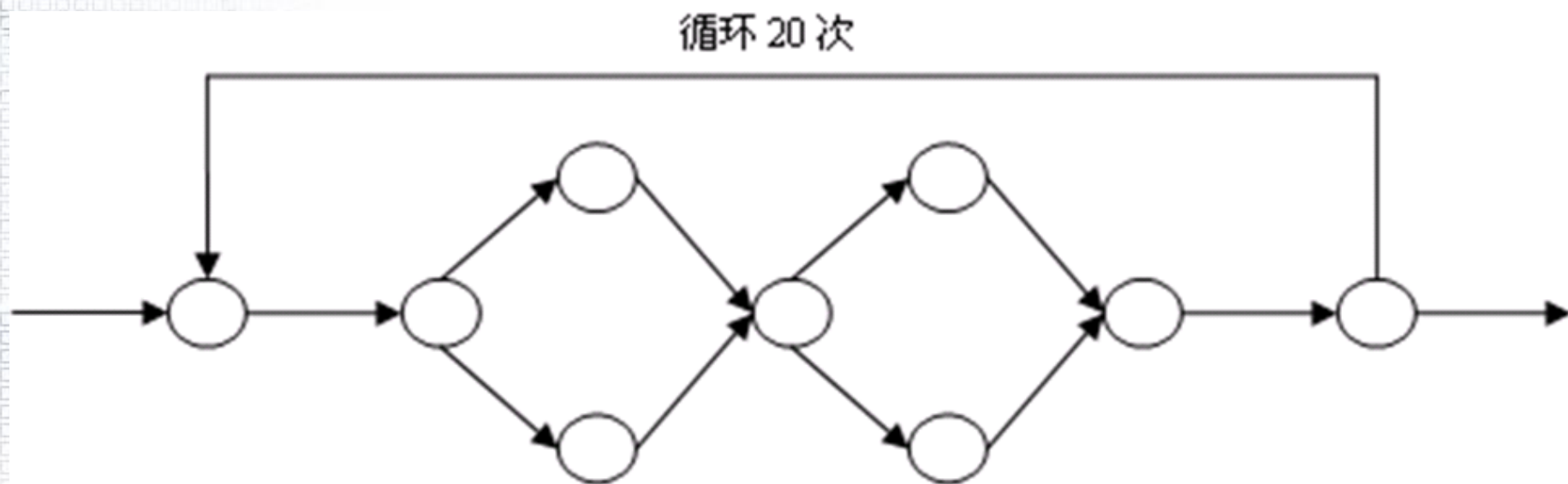
## 7.4.1 白盒测试概念

- 白盒测试（White Box）
  - 导出测试用例是依据模块的编码，即模块的内部逻辑对测试者是可见的
  - 只要测试了程序的所有路径，程序就应该是100%正确的？
  - 是否能穷尽所有路径？**实际上这是不可能的。**
  - 即使穷尽了路径，是否能保证测试的结果可靠？



## 7.4.1 白盒测试的概念

- 能否穷尽所有路径？



$$4^1 + 4^2 + \dots + 4^{20} \approx 1.466 \times 10^{12}$$





## 7.4.1 白盒测试的概念

—即使穷尽了路径，是否能保证测试的结果可靠？

```
if (x+y+z)/3 == x then  
    output("x, y, z的值相等")  
else  
    output("x, y, z的值不相等")  
end if
```

无法穷尽！  
即使穷尽，也未必可靠！

测试1: x=1, y=2, z=3

测试2: x=2, y=2, z=2

---

测试3: x=2, y=1, z=3



## 7.4.1 白盒测试的概念

- 白盒测试要保证
  - 模块中所有独立途径至少测试一次
  - 测试所有的逻辑决策真和假两个方面
  - 在所有循环的边界内部和边界上执行循环体
  - 检查内部数据结构以保证其有效性
- 测试策略
  - 选择代表性的路径
  - 选择测试数据TCS



## 7.4.2 基本途径测试

### • 概念

- 设计出的测试用例要保证在测试中程序的每一条可执行语句至少执行一次
- 基本途径的集合是由一组独立途径组成的
  - 独立途径是指程序中至少引入一条新（执行）语句的路径
  - 独立途径可以通过程序流程图和环形复杂度的概念来确定
    - 路径1: Start → A → B → End
    - 路径2: Start → A → C → End



## 7.4.2 基本途径测试

- 途径

- 1-2-10

- 1-2-3-4-6-8-9-2-10

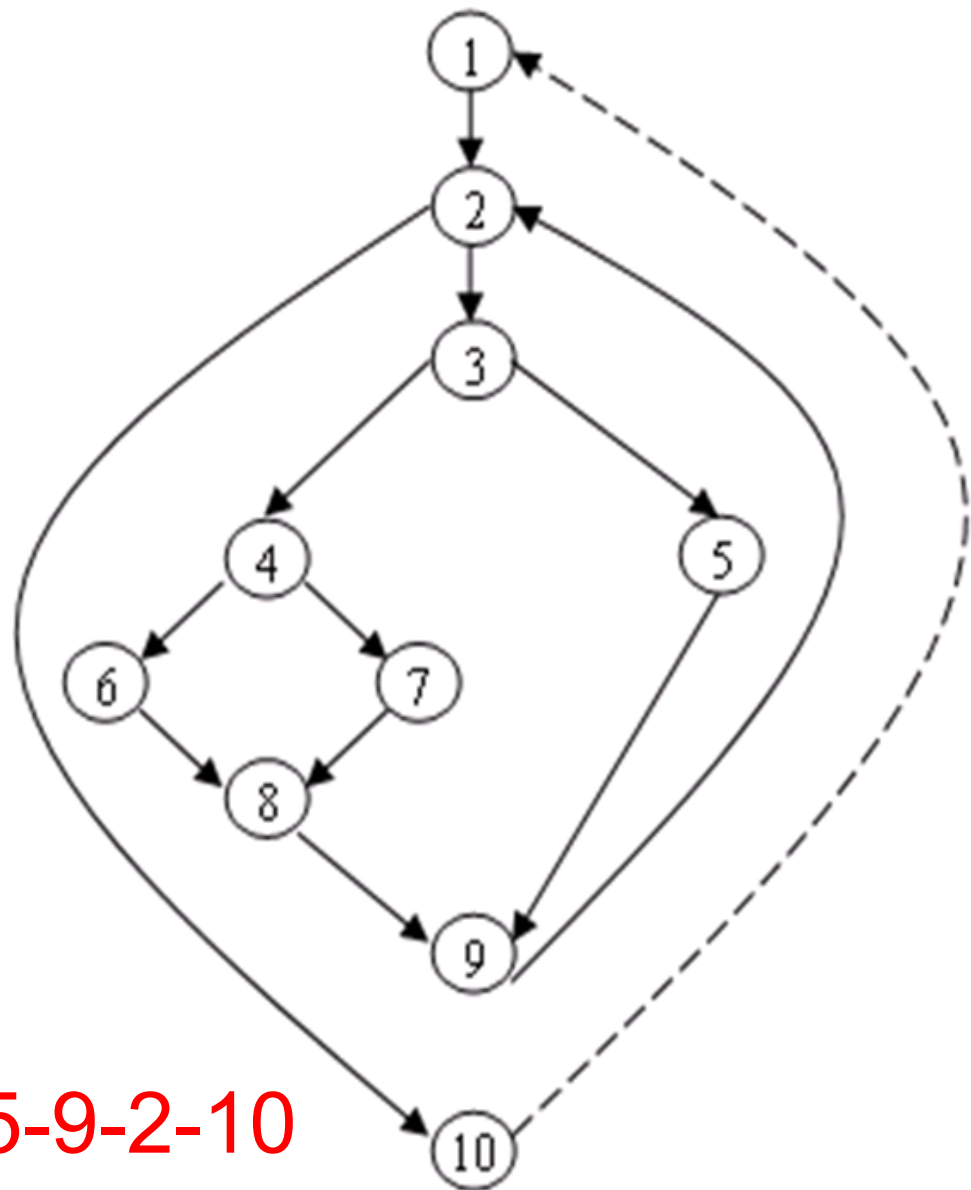
- 1-2-3-4-7-8-9-2-10

- 1-2-3-5-9-2-10

环形复杂度

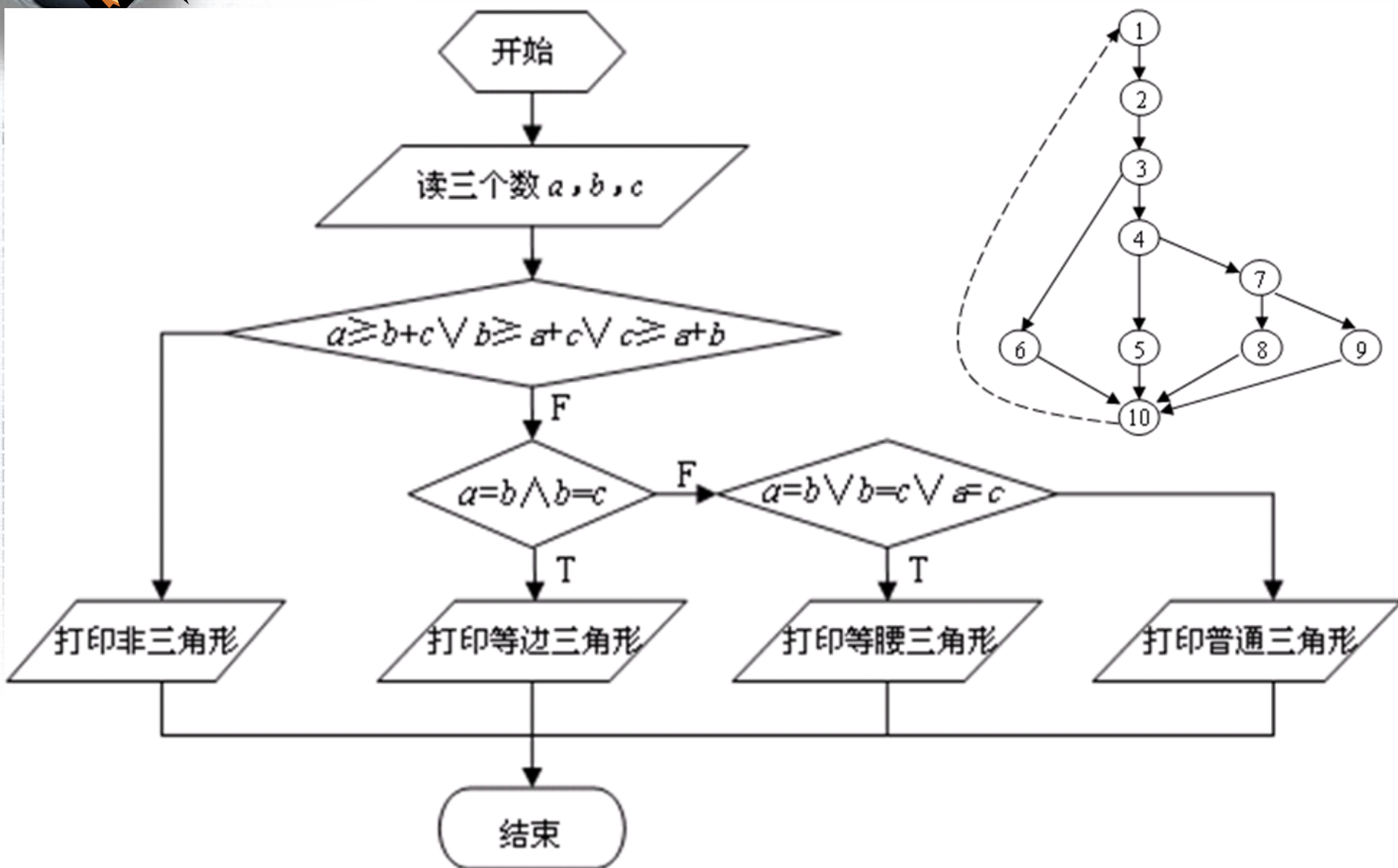
=判定节点数 +1;

- 1-2-3-4-6-8-9-2-3-5-9-2-10





## 7.4.2 基本途径测试





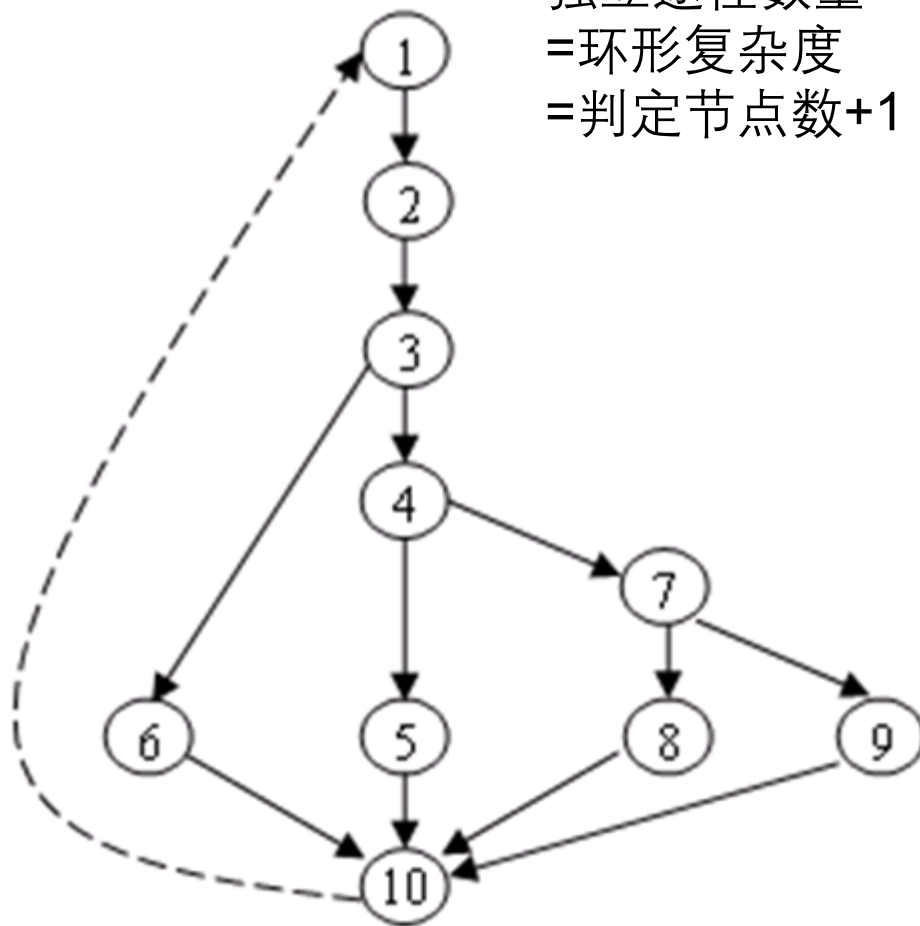
## 7.4.2 基本途径测试

- 途径

- 1-2-3-6-10
- 1-2-3-4-5-10
- 1-2-3-4-7-8-10
- 1-2-3-4-7-9-10

- 测试实例

- $a=5$ ,  $b=2$ ,  $c=2$
- $a=2$ ,  $b=2$ ,  $c=2$
- $a=2$ ,  $b=2$ ,  $c=3$
- $a=3$ ,  $b=4$ ,  $c=5$





## 7.4.2 基本途径测试

PROCEDURE averagy;

\*This procedure computes the averagy of 100 of fewer numbers that lie bounding values; it also computes the total input and the total vaitid.

INTERFACE RETURNS averagy, total. input, total, valid,

INTERFACE ACCEPTS vaiae, minimum, maximum;

TYPE value [1:100] IS SCALAR ARRAY;

TYPE averagy, total. input, total. valid, minimum, maximum, sum IS SCALAR;

TYPE i IS INTEGER;

i=1;

total. input=total. valid=0;

sum=0;

DO WHILE value[i] <>-999 AND total. input<100

    increment total. input by 1;

    IF value[i]>=minimum AND value[i]≤ maximum

        THEN increment total.valid by 1;

        sum=sum+value [i];

    ELSE skip

    ENDIF;

    increment i by 1;

ENDDO

IF total. valid>0

    THEN averagy=sum/total. valid;

    ELSE averagy=-999;

ENDIF

END averagy



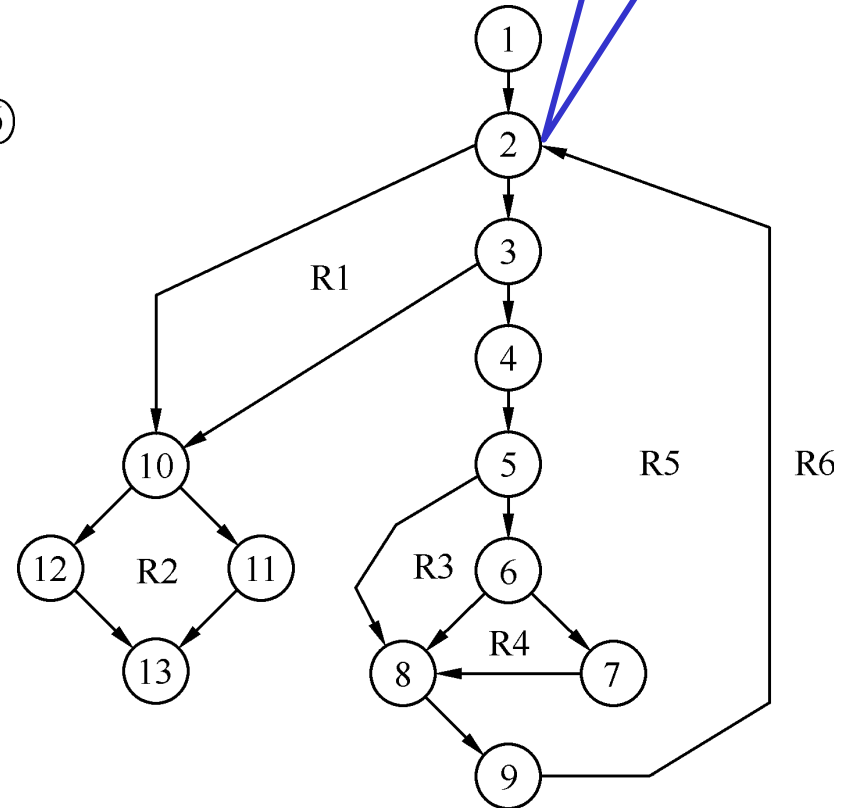
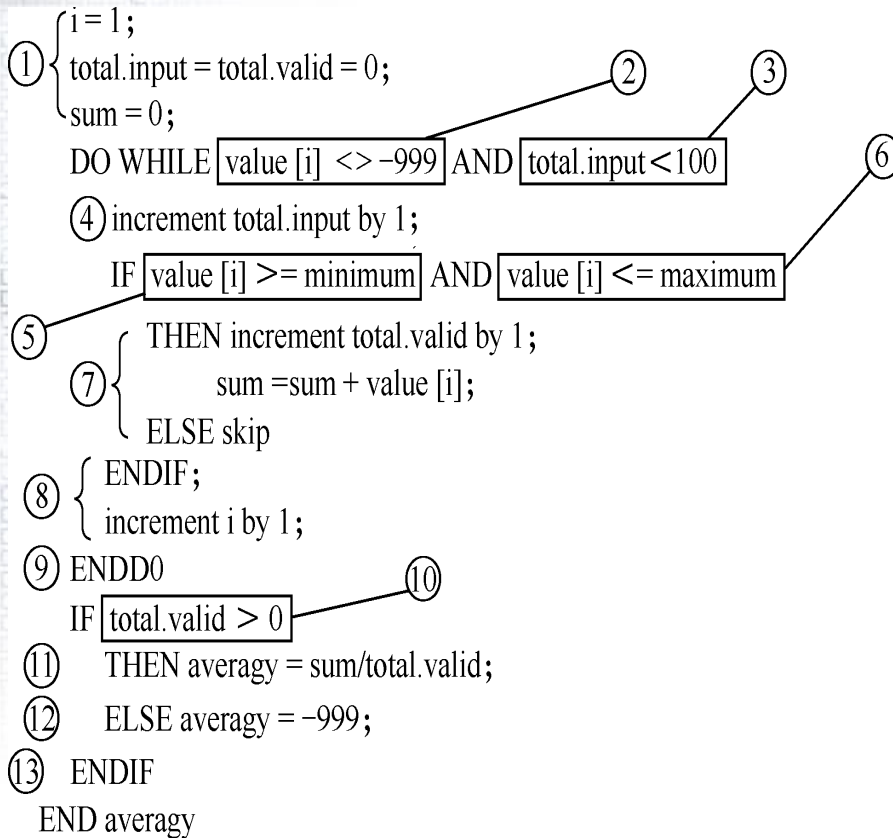


结点2、3、5、6和10都是判断节点。

# 基本途径测试

## 由过程描述导出控制流图

2、3、5、6、10  
都是判断节点



独立途径数量=环形复杂度=5(判定节点数)+1=6

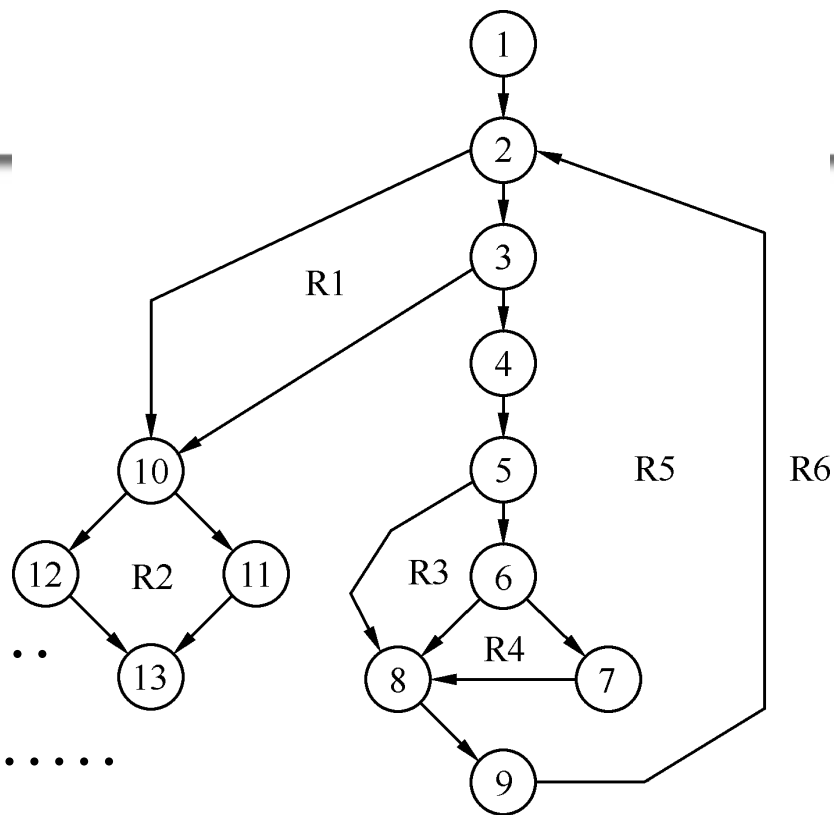


## 基本途径测试

- path1: 1-2-10-11-13
- path2: 1-2-10-12-13
- path3: 1-2-3-10-11-13
- path4: 1-2-3-4-5-8-9-2.....
- path5: 1-2-3-4-5-6-8-9-2.....
- path6: 1-2-3-4-5-6-7-8-9-2.....

路径4、5、6后面的省略号 (.....)

表示在控制结构中以后剩下的路径是可选的





## 7.4.3 条件测试

- 概念

- 检查程序中所包含的逻辑条件

- 简单条件

- 是一个布尔变量或者一个关系表达式，可以在它们前面有一个非操作（NOT）

$$E_1 < \text{关系算子} > E_2$$

- $E_1$ 和 $E_2$ ：算数表达式
      - 关系算子：“<”，“≤”，“≠”，“=”，“≥”和“>”



## 7.4.3 条件测试

### • 概念

#### — 组合条件

- 由两个或多个简单条件、布尔算子和括号构成
- 布尔算子：OR（“|”），AND（“&”），NOT（“¬”）

#### — 条件中包含的错误类型如下：

- 布尔算子错误
- 布尔变量错误
- 括号不匹配
- 关系算子错误
- 算术表达式错误



## 7.4.3 条件测试

- 分支和关系算子测试方法（BRO）
  - BRO（Branch and Relational Operator条件测试
  - 对于一个有 $n$ 个简单条件的条件 $C$ 的约束定义为 $(D_1, D_2, \dots, D_n)$ 
    - $D_i$  ( $1 \leq i \leq n$ ) 是规定了条件 $C$ 中第 $i$ 个简单条件输出的约束符号
  - 如果在条件 $C$ 的执行中，其每个简单条件的输出满足 $D$ 中对应的约束，则称 $C$ 的执行覆盖了 $C$ 的条件约束 $D$ 。



## 7.4.3 条件测试

- BRO条件测试是一种白盒测试技术，要求：
  - 1. 每个条件的每个可能结果至少执行一次
  - 2. 每个判定语句的每个可能结果至少执行一次
  - 3. 覆盖所有可能的条件组合结果



## 7.4.3 条件测试

- 例子1  $C_1:B_1 \& B_2$ 
  - 约束形式为  $(D_1, D_2)$
  - $\{ (t, t), (t, f), (f, t) \}$
- 例子2  $C_2:B_1 \& (E_3=E_4)$ 
  - 约束形式为  $(D_1, D_2)$
  - $\{ \underbrace{(t, =)}_{(t, t)}, \underbrace{(t, >), (t, <)}_{(t, f)}, \underbrace{(f, =)}_{(f, t)} \}$





## 7.4.3 条件测试

- 例子3  $C_3: (E_1 > E_2) \ \& \ (E_3 = E_4)$ 
  - 约束形式为  $(D_1, D_2)$
  - $\{(>, =), (>, >), (>, <), (=, =), (<, =)\}$ 
    - $(t, t)$
    - $(t, f)$
    - $(f, t)$
  - $(t, t) \rightarrow (>, =)$
  - $(t, f) \rightarrow (>, >), (>, <)$
  - $(f, t) \rightarrow (=, =), (<, =)$



## 7.4.3 条件测试

- 例子4 判断三角形类型的三个判定条件
  - $C_1: (a \geq b+c) \vee (b \geq a+c) \vee (c \geq a+b)$
  - 约束形式为  $(D_1, D_2, D_3)$
  - $\{ (t, f, f), (f, t, f), (f, f, t), (f, f, f) \}$
  - $(t, t, t) \times$
  - $C_2: (a=b) \wedge (b=c)$
  - 约束形式为  $(D_1, D_2)$
  - $\{ (t, t), (f, t), (t, f), (f, f) \}$
  - $C_3: (a=b) \vee (b=c) \vee (a=c)$
  - 同  $C_1$



## 7.4.3 条件测试

- 例子4
  - 条件  $C_1$ :

| 测试用例 | a | b | c | $a > b + c$<br>$D_1$ | $b > a + c$<br>$D_2$ | $c > a + b$<br>$D_3$ | $C_1$ | 覆盖的组合     |
|------|---|---|---|----------------------|----------------------|----------------------|-------|-----------|
| 1    | 5 | 2 | 2 | T                    | F                    | F                    | T     | (T, F, F) |
| 2    | 2 | 5 | 2 | F                    | T                    | F                    | T     | (F, T, F) |
| 3    | 2 | 2 | 5 | F                    | F                    | T                    | T     | (F, F, T) |
| 4    | 2 | 3 | 4 | F                    | F                    | F                    | F     | (F, F, F) |



## 7.4.3 条件测试

- 例子4
  - 条件 $C_2$ :

| 测试用例 | a | b | c | $a=b(D_1)$ | $b=c(D_2)$ | $C_2$ | 覆盖的组合  |
|------|---|---|---|------------|------------|-------|--------|
| 1    | 2 | 2 | 2 | T          | T          | T     | (T, T) |
| 2    | 2 | 2 | 3 | T          | F          | F     | (T, F) |
| 3    | 2 | 3 | 3 | F          | T          | F     | (F, T) |
| 4    | 3 | 4 | 5 | F          | F          | F     | (F, F) |



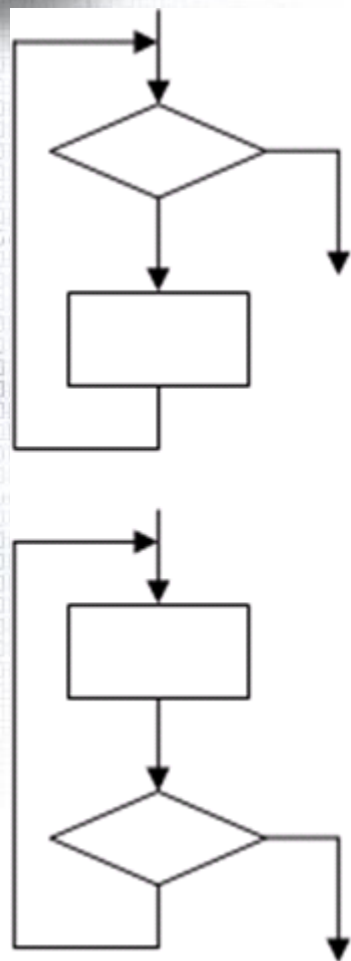
## 7.4.3 条件测试

- 例子4
  - 条件 $C_3$ :

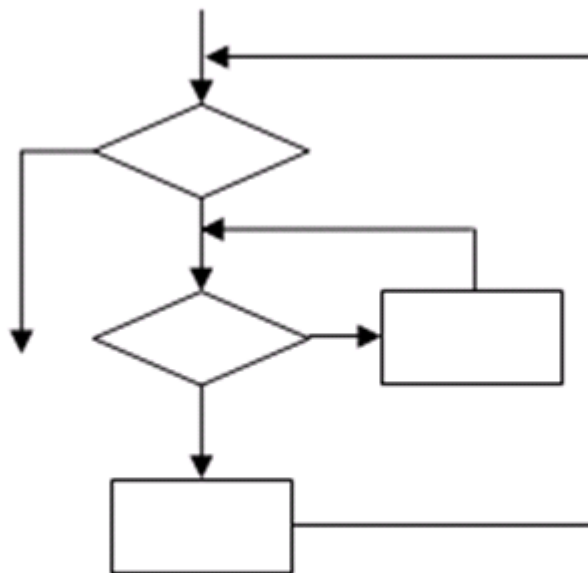
| 测试用例 | a | b | c | a=b<br>$D_1$ | b=c<br>$D_2$ | a=c<br>$D_3$ | $C_3$ | 覆盖的组合     |
|------|---|---|---|--------------|--------------|--------------|-------|-----------|
| 1    | 2 | 2 | 3 | T            | F            | F            | T     | (T, F, F) |
| 2    | 2 | 3 | 2 | F            | F            | T            | T     | (F, F, T) |
| 3    | 3 | 2 | 2 | F            | T            | F            | T     | (F, T, F) |
| 4    | 3 | 4 | 5 | F            | F            | F            | F     | (F, F, F) |



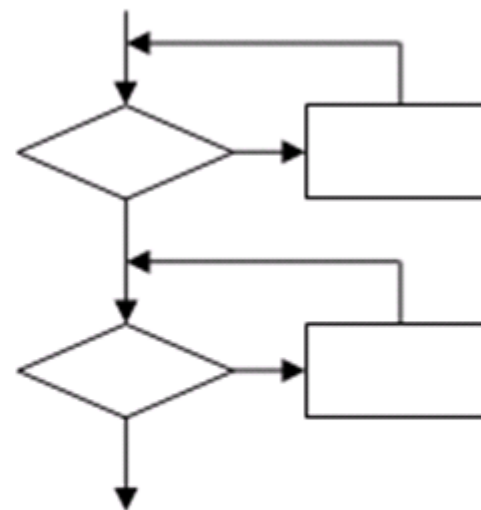
## 7.4.4 循环测试



简单循环



嵌套循环



级联循环



## 7.4.4 循环测试

- 简单循环
  - $n$ 为允许通过循环的最大次数
  - 1)完全跳过该循环
  - 2)仅仅通过循环体一次
  - 3)通过循环体二次
  - 4)通过循环体 $m$ 次, 其中 $m < n$
  - 5)通过循环体 $n-1$ 次,  $n$ ,  $n+1$ 次





## 7.4.4 循环测试

- 嵌套循环
  - 1) 从最内层循环开始，把所有外层循环置为最小值
  - 2) 将外层循环控制在最小循环参数上，对内层循环进行简单循环测试。然后再为外层控制参数的某些值增加其它的测试
  - 3) 向外加工，为下一个循环进行测试
  - 4) 继续进行，直至所有的循环测试完成



## 7.4.4 循环测试

- 级联循环
  - 通常使用简单循环测试方法分别进行
  - 如果这两个级联循环是相关的，如将第一个循环的结束循环参数作为第二个循环的开始参数，或者是某些状态变量互相关联，则应该使用嵌套循环的测试方法



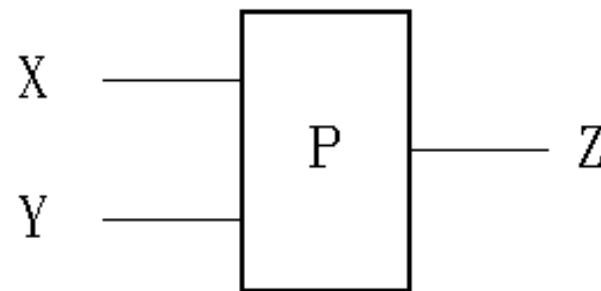
## 7.5 黑盒测试

- 黑盒测试概念

- 在程序或模块的接口级进行，而不考虑程序的内部逻辑
- 用于检测程序中下列类型的错误
  - 不正确或漏掉的功能
  - 接口错误
  - 数据结构或外部数据库存取中的错误
  - 初始化或结束错误
  - 性能方面的问题



## 7.5.1 概念



- 假设一个程序**P**有输入量**X**和**Y**及输出量**Z**。在字长为32位的计算机上运行。若**X**、**Y**取整数，按黑盒方法进行穷举测试，可能采用的测试数据组：  
$$2^{32} \times 2^{32} = 2^{64}$$
- 如果测试一组数据需要1毫秒，一年工作  $365 \times 24$  小时，完成所有测试需5亿年。



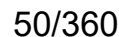
## 7.5.1 等价类划分

- 概念

- 把一个程序输入的定义域划分成不同的数据类，然后根据这些数据类可以导出测试用例
- 等价类是由相对于程序的功能具有相同作用的一些输入数据元素构成的数据集合，这些数据元素之间具有等价关系
- 利用等价类中一个元素作为代表对程序进行测试，而不是使用该类的全体成员，藉此以减少总的测试用例数量



- 如果输入条件规定了一个范围[a, b], 则可以得到一个有效等价类和两个无效等价类
  - 一个有效等价类:  $\{ x \mid x \in [a, b] \}$
  - 两个无效等价类:  $\{ x \mid x < a \}$ 、 $\{ x \mid x > b \}$
- 例如, 在程序的规格说明中, 对输入数据有一句话: “.....项数可以从1到999.....”, 则有效等价类是 “ $1 \leq \text{项数} \leq 999$ ”, 两个无效等价类是 “项数 $<1$ ”及 “项数 $>999$ ”。在数轴上表示为:







## 7.5.1 等价类划分

- 如果输入条件规定的是某集合S的一个成员a
  - 一个有效等价类:  $\{ x \mid x=a \}$
  - 一个无效等价类:  $\{ x \mid x \neq a \}$
  - 例如: “.....本地人须.....”, 则考虑本地人为一有效等价类; 非本地人为无效等价类
- 如果输入条件规定了一个特殊的值, 如 “#”
  - 一个有效等价类:  $\{ x \mid x \text{以} \# \text{开头} \}$
  - 一个无效等价类:  $\{ x \mid x \text{不以} \# \text{开头} \}$
  - 例, 在C语言中对变量标识符规定为 “以字母“#”打头的.....串”
- 如果输入要求是布尔值t
  - 一个有效等价类:  $\{ x \mid x==t \}$
  - 一个无效等价类:  $\{ x \mid x \neq t \}$





## 7.5.1 等价类划分

- 例子1：数据库产品规范要求该产品必须能够处理从1到65535 ( $2^{16}-1$ )
  - 等价类1：少于一个记录
  - 等价类2：从1到65535个记录
  - 等价类3：多于65535 个记录
- 例子2：判断三角形的程序
  - 等价类1：等边三角形
  - 等价类2：等腰三角形
  - 等价类3：普通三角形
  - 等价类4：非三角形



## 7.5.2 边界值分析

- 等价类分析技术的补充
- 专门选择等价类“边”上的元素
- 边界值分析的指导原则：
  - 如果输入条件规定了由值a和值b界定的范围
    - a-1、a、a+1、b-1、b、b+1、以及a和b中间的值
  - 如果输入条件规定了一些数值
    - 最大数、最小数、这些数值、以及刚好大于、刚好小于最大与最小数的一些值
  - 如果一个内部数据结构已经规定了边界
    - 相当于默认了一个范围
  - 同理适用于输出条件



## 7.5.2 边界值分析

- 例1：如果输入条件规定类由值a和b界定的范围，则测试用例应包括7组：
  - a, 刚刚小于a, 刚刚大于a
  - b, 刚刚小于b, 刚刚大于b
  - [a, b]之间的



## 7.5.2 边界值分析

- 例2：数据库产品规范要求该产品必须能够处理从1到65535 ( $2^{16}-1$ )

— 测试用例1：0个记录

} 等价类1：  
少于1条记录

— 测试用例2：1个记录

— 测试用例3：2个记录

— 测试用例4：100个记录

— 测试用例5：65534个记录

— 测试用例6：65535个记录

} 等价类2：  
从1到65535个记录

— 测试用例7：65536个记录

} 等价类3：  
多于65535个记录



## 7.5.2 边界值分析

### • 例3：判断三角形程序

- 等价类1：等边三角形
- 等价类2：等腰三角形
- 等价类3：普通三角形
- 等价类4：非三角形

| 用例编号 | 输入 (a,b,c) | 所属等价类 | 类型说明      | 预期输出    |
|------|------------|-------|-----------|---------|
| 1    | 1,1,1      | 等价类1  | 等边三角形边界   | 等边三角形   |
| 2    | 3,3,4      | 等价类2  | 等腰三角形     | 等腰三角形   |
| 3    | 2,2,3      | 等价类2  | 等腰三角形边界   | 等腰三角形   |
| 4    | 3,4,5      | 等价类3  | 普通三角形     | 一般三角形   |
| 5    | 2,3,4      | 等价类3  | 普通三角形边界   | 一般三角形   |
| 6    | 1,2,3      | 等价类4  | 两边和 = 第三边 | 不能构成三角形 |
| 7    | 0,2,3      | 等价类4  | 含零边长      | 不能构成三角形 |
| 8    | -1,3,4     | 等价类4  | 含负边长      | 不能构成三角形 |



## 7.6 单元测试

- 概念 (unit testing)
  - 把一个模块作为独立的程序单元进行测试，以保证它能够正确执行规定的功能
  - 黑盒测试方法与白盒测试方法都适用于单元测试，它们是相互补充的，但不能相互代替



## 7.6 单元测试

- 单元测试的考虑

- 模块接口

- 参数个数、类型、顺序、形实参匹配、输入输出合法

- 模块执行外部I/O操作

- 文件读写、数据库读写、网络请求、外设输入输出、调用外部设备或接口的合法性与异常情况

- 模块的局部数据结构

- 变量初始化、数组越界、数据溢出

- 模块的计算

- 分支、循环、逻辑运算，循环上下限、极值、空值、0 值

- 模块的错误处理例程

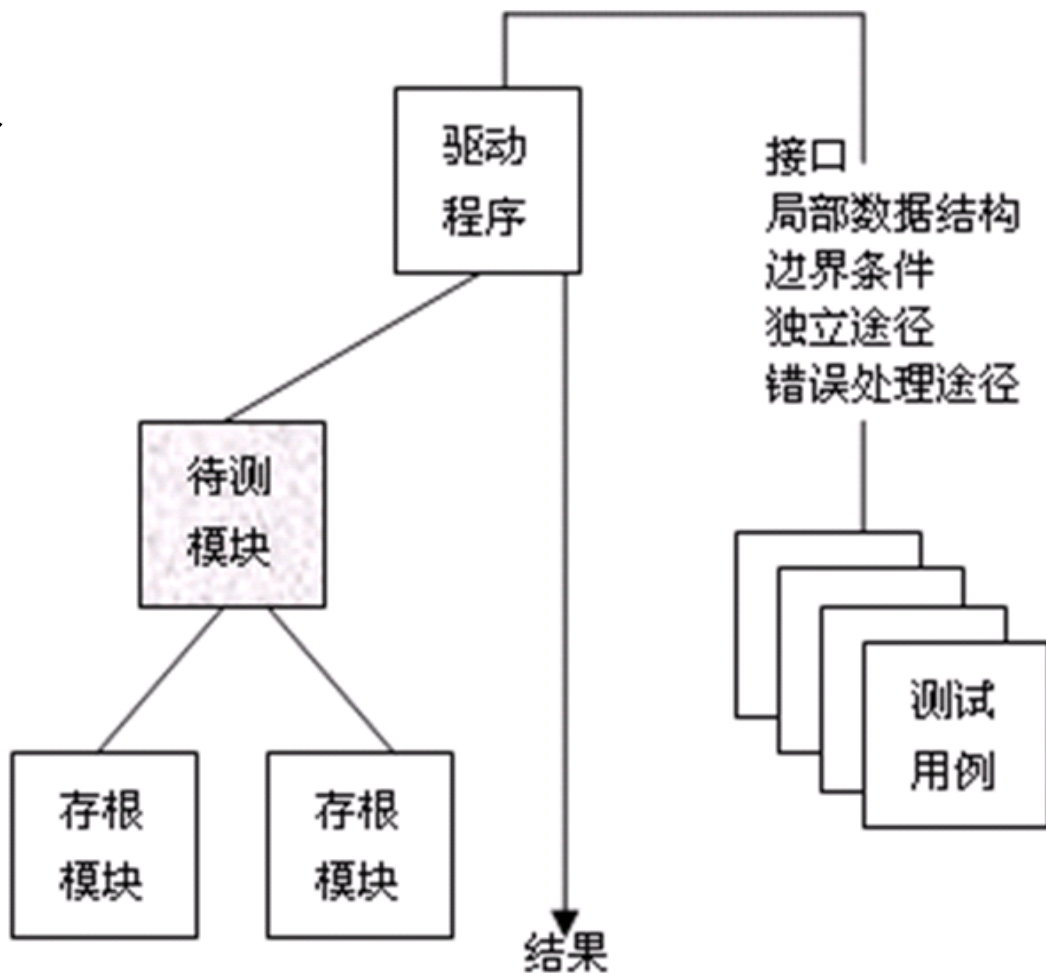
- 异常捕获、报错提示、容错





## 7.6 单元测试

- 驱动程序的实现
- 存根模块的实现
- 测试开销





## 7.7 集成测试

- 概念

- 将已经通过彻底测试的模块组装起来，以形成一个系统或软件产品，对其进行测试
- 主要使用黑盒测试法
- 任务是主要是检查和排除
  - 模块间接口错误
  - 全局数据结构错误
  - 模块中某些遗漏的错误，
  - 系统的功能和性能是否满足规范要求



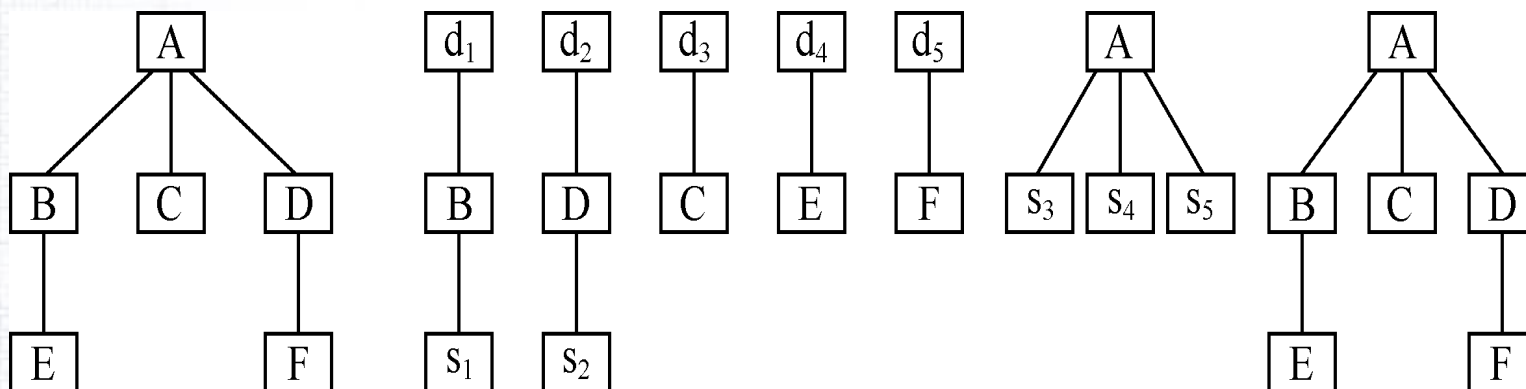
## 7.7 集成测试

- 集成测试方法
  - 一次性集成测试
    - 先对每个模块分别测试，再把所有模块组装在一起进行测试
  - 增量集成测试
    - 先对每个模块进行测试，然后一次集成一个模块，增值组装测试
      - 自顶向下集成测试
      - 自底向上集成测试
      - 三明治测试



## 7.7 集成测试

- 一次性集成示例
  - 先分别单元测试，再统一集成



- 优点：简单、效率高、工作量小
- 缺点：难以定位错误



## 7.7 集成测试

- 增量集成测试-自顶向下集成

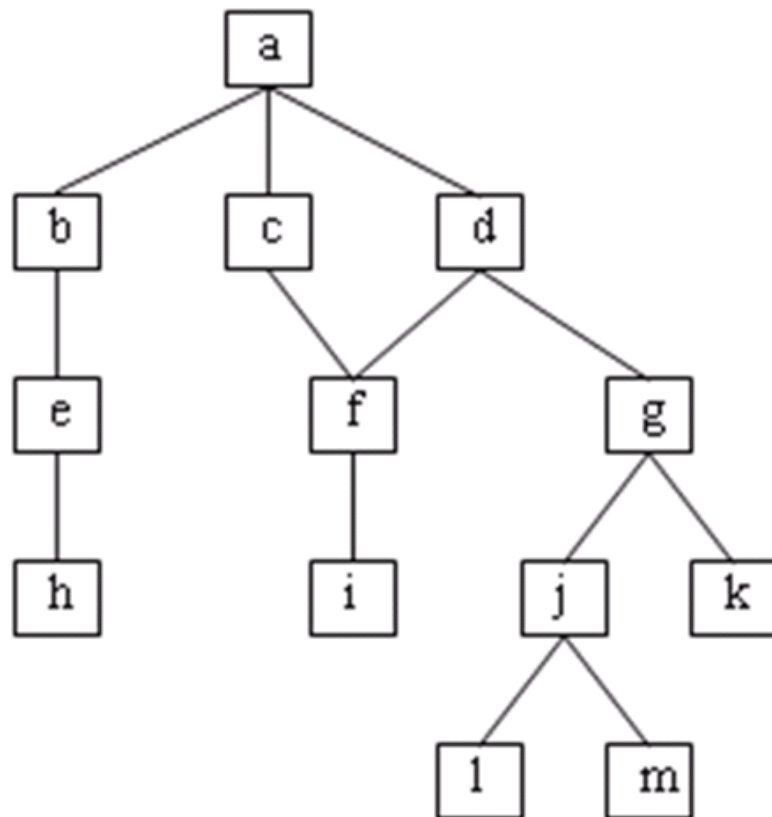
- 从主控模块开始，沿着控制层向下移动测试

- 深度优先

- a, b, e, h, c, f,  
i, d, g, j, l, m和k

- 宽度优先

- a, b, c, d, e, f,  
g, h, i, j, k, l和m





## 7.7 集成测试

- 自顶向下集成测试

- 优点:

- 早期发现主要设计错误
    - 容易孤立错误
    - 不需要驱动模块

- 缺点:

- 需要构建存根模块
    - 潜在的、可再使用的模块可能没有得到充分的测试
    - 工作量大



## 7.7 集成测试

- 自底向上集成测试

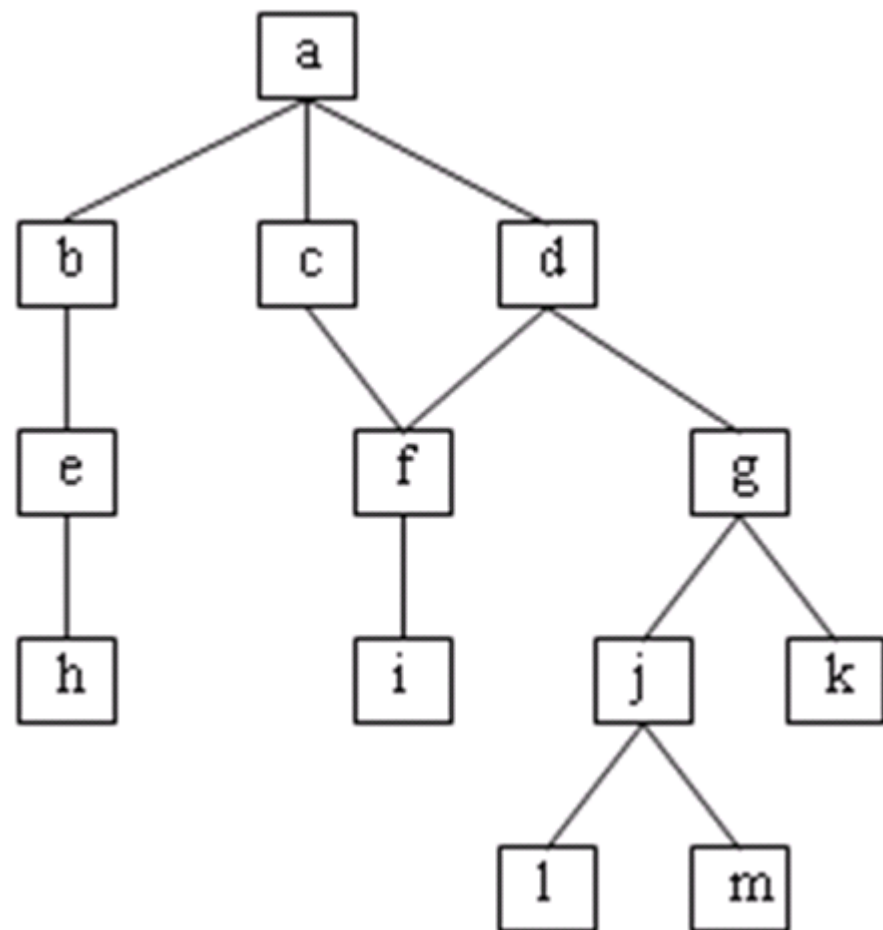
- 从底层的模块开始测试，沿着控制层向上，一次结合一个模块

- 可能的顺序1

- l, m, h, i, k, e, f, j, g, b, c, d和a

- 可能的顺序2

- h, e, b, i, f, c, l, m, j, k, g, d和a







## 7.7 集成测试

- 自底向上集成测试

- 优点:

- 容易孤立错误
    - 减少存根模块带来的开销

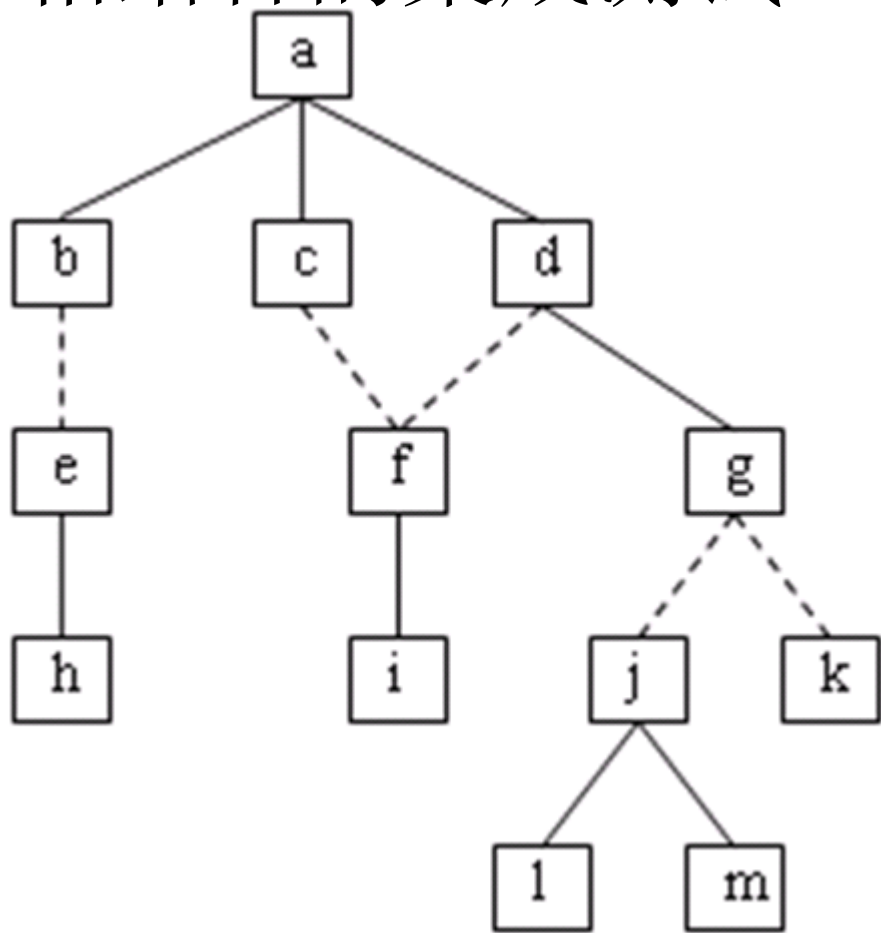
- 缺点:

- 后期才发现设计错误，引发大量程序修改
    - 工作量大



## 7.7 集成测试

- 自顶向下和自底向上相结合的集成测试
  - 逻辑模块
    - a, b, c, d和g
  - 操作模块
    - h, e, i, f, l, m, j和k
  - 优点
    - 仍可孤立错误





## 7.7 集成测试

- 回归测试（Regression Test）
  - 集成测试中，软件发生修改（如修复缺陷、添加新功能、修改现有功能等）后可能会引入其它错误，因此必须重新进行测试，防止由于改动错误引发的副作用
  - 并不是重新执行所有的测试用例
    - 1) 执行测试所有软件功能的代表性测试用例
    - 2) 测试可能受改变影响的那些功能的测试用例
    - 3) 测试已经改变的软件部件的测试用例
    - Why? 开销大+测试用例规模大



## 7.8 产品交付阶段测试

- 其他测试

- 系统测试

- 恢复测试（recovery testing）
    - 安全保密性测试（security testing）
    - 压力测试（stress testing）
    - 性能测试（performance testing）

- $\alpha$ 测试

- $\beta$ 测试

- 验收测试

需求分析 → 开发 → 单元测试 → 集成测试 → 系统测试 → **Alpha**测试 → **Beta**测试 → 验收测试 → 正式发布



## 7.9 相关文档规范

- 相关测试文档
  - 1) 软件工程实践者的研究方法
  - 2) RUP:
    - 测试计划
    - 测试用例
    - 测试评估摘要
  - 3) 国家标准：软件测试报告的编写内容
  - 4) 企业标准



## 7.9 软件测试文档

### • 示例

|       |                                                                                                                                                                       |
|-------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 用例名称  | 注册会员测试用例                                                                                                                                                              |
| 用例 id | C-001                                                                                                                                                                 |
| 基本描述  | 用户输入用户基本信息，在数据库生成相应数据记录。                                                                                                                                              |
| 测试方案  | 测试正确输入、用户名为空、密码与密码不相同、年龄不数字、年龄不在正确范围内、电话不为数字、信箱地址不符合规范等情况。                                                                                                            |
| 输入数据  | 1. → 输入正确数据。<br>2. → 没有输入用户名。<br>3. → 密码与密码确认不同，分别为 <u>selley</u> 和 <u>salley</u> 。<br>4. → 年龄项输入 a<br>5. → 年龄项输入 200<br>6. → 电话输入 02412345678<br>7. → 信箱地址输入 myy.com |
| 预期结果  | 第一组测试正确执行，注册新用户成功。<br>第二组测试系统提示没有输入用户名。<br>第三组测试系统提示密码输入与密码确认不同。<br>第四组测试系统提示年龄要为数字。<br>第五组测试系统提示年龄不在有效范围内。<br>第六组测试系统提示电话号码无效。<br>第七组测试系统提示信箱地址格式不正确。                |

测试用例 = {执行条件 + 测试数据 + 期望结果} + 实际结果

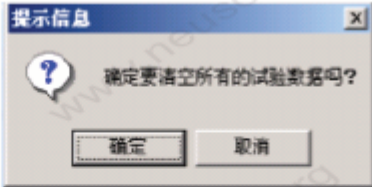
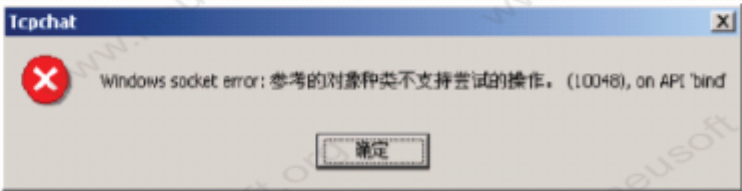




# 7.9 软件测试文档

## • 示例

XX项目测试报告

| 序号 | 功能          | 错误描述                                                                                 |         |            | 统计项   |     |           |           |    |
|----|-------------|--------------------------------------------------------------------------------------|---------|------------|-------|-----|-----------|-----------|----|
|    |             | 应达目标                                                                                 | 实际结果    | 出现错误条件(几率) | 错误类型  | 测试者 | 测试日期      | 改正日期      | 状态 |
| 1  | 综合查询        | 窗口标题为中文                                                                              | 窗口标题为英文 |            | 文字标识错 | XX  | 2004-2-20 | 2004-2-21 | 完成 |
|    |             |     |         |            |       |     |           |           |    |
| 2  | 综合查询        | 查询条件字段为中文名                                                                           | 英文      |            | 文字标识错 | XX  | 1900-1-1  |           | 待改 |
| 3  | 系统维护—清空实验数据 | 有警告, 并且默认按钮为取消                                                                       | 默认按钮为放弃 |            | 数据安全类 | XX  | 1900-1-10 |           | 否决 |
|    |             |    |         |            |       |     |           |           |    |
| 4  | 通讯客户端       | 未能和服务端连接时, 做提示                                                                       | 提示不够用户化 |            | 系统异常  |     |           | 1900-1-30 | 待改 |
|    |             |  |         |            |       |     |           |           |    |